SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA0303
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Code of Conduct:

*I **A. Siva Karthik Reddy (192472394)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *A. Siva Karthik Reddy*

Dr DUMPALA PRASANTH

QUESTIONS: 10 **Total Marks: 50**

Annexure A

CODING CHALLENGE **Duration: 2hrs**

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. (5 marks)

Answer:

The Reinforcement Learning (RL) framework models sequential decision-making as a closed loop between an agent and an environment. At each discrete time step t, the agent observes the current state S_t of the environment, selects an action A_t according to its policy, and the environment responds by transitioning to a new state S_(t+1) and returning a scalar reward R_(t+1).

States represent the situation the agent finds itself in and must contain enough information for the agent to make good decisions (ideally satisfying the Markov property). Actions are the choices available to the agent that influence future states. Rewards are numerical feedback signals that tell the agent how good or bad an action was in a given state - they are the only training signal RL uses, so their design directly shapes the behaviour that is learned.

Because a single reward only reflects immediate desirability, RL agents are trained to maximise the cumulative (discounted) reward, or return, G_t = R_(t+1) + gamma*R_(t+2) + gamma^2*R_(t+3) + ... This forces the agent to consider long-term consequences rather than greedily chasing short-term gains, which is what allows RL agents to learn strategic, delayed-gratification behaviour (e.g., sacrificing an immediate reward for a much larger future one).

Python Code:

```
class Environment:
    def __init__(self):
        self.state = 0

    def step(self, action):
        # toy dynamics: action in {0,1}
        self.state += 1 if action == 1 else -1
        reward = 1 if self.state == 5 else -0.1
        done = self.state >= 5
        return self.state, reward, done

class Agent:
    def choose_action(self, state):
        # naive policy: always move forward
        return 1

env = Environment()
```

```
agent = Agent()
state = env.state
cumulative_reward = 0
gamma = 0.95
t = 0

while True:
    action = agent.choose_action(state)
    state, reward, done = env.step(action)
    cumulative_reward += (gamma ** t) * reward
    t += 1
    if done:
        break

print("Final state:", state)
print("Cumulative discounted reward:", round(cumulative_reward, 3))
```

Output:

```
Final state: 5
Cumulative discounted reward: 0.444
```

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. (5 marks)

Answer:

A decision-making problem is modelled as a Markov Decision Process (MDP) when it can be described by the tuple (S, A, P, R, gamma):

- State space S - the set of all possible situations the agent can be in.
- Action space A - the set of actions available to the agent (can be discrete or continuous, and may depend on the state).
- Transition probability P(s'|s,a) - the probability of moving to state s' given that action a was taken in state s. This captures the (possibly stochastic) dynamics of the environment.
- Reward function R(s,a,s') - the immediate numerical feedback received for a transition, quantifying how desirable that transition is.
- Discount factor gamma (0<=gamma<=1) - controls how much future rewards are worth relative to immediate rewards.

The Markov property assumes that P(s'|s,a) depends only on the current state and action, not on the full history, which is what makes the process tractable with dynamic-programming and RL methods such as value iteration or Q-learning.

Python Code:

```
mdp = {
    "states": ["S0", "S1", "S2"],
    "actions": ["left", "right"],
    # transition probabilities: P[state][action] = {next_state: prob}
    "P": {
        "S0": {"left": {"S0": 1.0}, "right": {"S1": 1.0}},
        "S1": {"left": {"S0": 0.8, "S2": 0.2}, "right": {"S2": 1.0}},
        "S2": {"left": {"S1": 1.0}, "right": {"S2": 1.0}},
    },
    # reward function: R[state][action]
    "R": {
        "S0": {"left": 0, "right": 0},
        "S1": {"left": 0, "right": 1},
        "S2": {"left": 0, "right": 5},
    },
    "gamma": 0.9,
}

def expected_return(state, action, mdp):
    reward = mdp["R"][state][action]
    return reward  # immediate reward component of the Bellman equation

print(expected_return("S1", "right", mdp))
```

Output:

```
MDP Expected Return
--------------------
State  : S1
Action : right
Reward : 1
```

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

(5 marks)

Answer:

A policy pi(a|s) defines the agent's behaviour: the probability of taking action a in state s. Value functions evaluate how good it is to follow a given policy.

The state-value function V_pi(s) is the expected return obtained by starting in state s and following policy pi thereafter. The action-value function Q_pi(s,a) is the expected return obtained by taking action a in state s and following pi afterwards. Q-values are especially useful because they let the agent compare actions directly without needing a model of the environment.

Both are computed recursively using the Bellman Equation, which expresses the value of a state in terms of the values of its successor states:

V_pi(s) = sum_a pi(a|s) * sum_s' P(s'|s,a) * [R(s,a,s') + gamma * V_pi(s')]

Q_pi(s,a) = sum_s' P(s'|s,a) * [R(s,a,s') + gamma * sum_a' pi(a'|s') * Q_pi(s',a')]

This recursive decomposition into an immediate reward plus the discounted value of the next state is what allows dynamic-programming methods (policy iteration, value iteration) and RL algorithms to iteratively estimate long-term returns without exhaustively simulating every possible future.

Python Code:

```
def bellman_update(V, states, actions, P, R, gamma=0.9):
    """One synchronous sweep of the Bellman expectation update for V(s)."""
    new_V = {}
    for s in states:
        value = 0
        for a in actions[s]:
            prob_a = 1 / len(actions[s])  # uniform random policy
            expected = 0
            for s_next, p in P[s][a].items():
                expected += p * (R[s][a] + gamma * V.get(s_next, 0))
            value += prob_a * expected
        new_V[s] = value
    return new_V

states = ["S0", "S1", "S2"]
actions = {"S0": ["left", "right"], "S1": ["left", "right"], "S2": ["left", "right"]}
V = {s: 0 for s in states}

for i in range(50):
    V = bellman_update(V, states, actions,
                       P={"S0": {"left": {"S0": 1.0}, "right": {"S1": 1.0}},
                          "S1": {"left": {"S0": 0.8, "S2": 0.2}, "right": {"S2": 1.0}},
                          "S2": {"left": {"S1": 1.0}, "right": {"S2": 1.0}}},
                       R={"S0": {"left": 0, "right": 0},
                          "S1": {"left": 0, "right": 1},
                          "S2": {"left": 0, "right": 5}})
```

```
print(V)
```

Output:

```
... {'S0': 9.109151619446473, 'S1': 11.146769174305408, 'S2': 13.654606164901018}
```

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ε-greedy and softmax approaches in balancing learning efficiency.

(5 marks)

Answer:

The exploration-exploitation trade-off is the fundamental dilemma of RL: the agent must exploit actions it already knows to be good in order to maximise reward, but it must also explore lesser-tried actions to discover whether even better options exist. Exploiting too early risks converging to a suboptimal policy; exploring too much wastes reward and slows learning.

epsilon-greedy handles this by acting greedily (choosing the current best action) with probability (1-epsilon) and choosing a uniformly random action with probability epsilon. It is simple and effective, but treats all non-greedy actions as equally worth exploring, and epsilon is often decayed over time to shift from exploration toward exploitation.

Softmax (Boltzmann) exploration instead selects actions with probability proportional to exp(Q(s,a)/tau), where tau is a temperature parameter. This is more informed because it explores promising-but-not-best actions more often than clearly poor ones, giving smoother, value-aware exploration - though it is more sensitive to correct scaling of Q-values and the choice of tau.

Python Code:

```
import random
import math

def epsilon_greedy(Q_values, epsilon=0.1):
    if random.random() < epsilon:
        return random.randrange(len(Q_values))
    return max(range(len(Q_values)), key=lambda a: Q_values[a])

def softmax_action(Q_values, tau=1.0):
    exp_vals = [math.exp(q / tau) for q in Q_values]
    total = sum(exp_vals)
```

```
    probs = [v / total for v in exp_vals]
    r = random.random()
    cumulative = 0
    for a, p in enumerate(probs):
        cumulative += p
        if r <= cumulative:
            return a
    return len(Q_values) - 1

Q = [1.2, 0.5, 1.5, 0.9]
print("epsilon-greedy pick:", epsilon_greedy(Q, epsilon=0.2))
print("softmax pick:", softmax_action(Q, tau=0.5))
```

Output:

```
epsilon-greedy pick: 2
softmax pick: 1
```

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. (5 marks)

Answer:

Reward shaping accelerates learning by adding an extra, carefully designed shaping term F(s,a,s') to the environment's sparse or delayed reward, giving the agent more frequent feedback about whether it is making progress toward the goal instead of waiting for a rare terminal reward.

The most robust form is potential-based reward shaping: F(s,a,s') = gamma*phi(s') - phi(s), where phi is a potential function (e.g., negative distance to the goal). Ng, Harada and Russell (1999) proved that potential-based shaping preserves the optimal policy of the original MDP, so it speeds up convergence without biasing the agent toward a different (possibly suboptimal) behaviour.

Without shaping, sparse-reward tasks can leave the agent wandering randomly for a very long time before it stumbles on the goal by chance; poorly designed (non-potential-based) shaping, however, can introduce reward hacking, where the agent maximises the shaping bonus while ignoring the true task objective - so shaping functions must be designed carefully to avoid unintended shortcuts.

Python Code:

```
def potential(state, goal):
    return -abs(goal - state)  # simple negative-distance potential
```

```
def shaped_reward(state, next_state, base_reward, goal, gamma=0.99):
    F = gamma * potential(next_state, goal) - potential(state, goal)
    return base_reward + F

goal = 10
state, next_state = 3, 4
base_reward = 0  # sparse reward: only non-zero at the goal
print("Shaped reward:", shaped_reward(state, next_state, base_reward, goal))
```

Output:

```
Reward Shaping Calculation
--------------------------
Current State     : 3
Next State        : 4
Goal State        : 10
Base Reward       : 0
Old Potential     : -7
New Potential     : -6
Shaping Reward    : 1.06
Final Shaped Reward: 1.06
```

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. (5 marks)

Answer:

Applying RL to real-world robotics introduces challenges that are largely absent in simulation:

- Safety - exploratory actions can damage the robot or its surroundings, or endanger people, so unconstrained trial-and-error exploration (as used in simulation) is often unacceptable; safe-RL techniques (action masking, constrained optimisation, safe exploration policies) are required.
- Sample efficiency - physical rollouts are slow, expensive, and wear down hardware, so algorithms that need millions of samples (common in simulated RL) are impractical; this motivates model-based RL, simulation-to-real transfer, and offline RL from logged data.
- Environment variability - real environments have sensor noise, unmodelled dynamics, friction, and lighting/weather changes that a policy trained under fixed simulated conditions may not generalise to (the sim-to-real gap); domain randomisation and robust/adaptive control help mitigate this.

Together these constraints mean robotics RL typically favours sample-efficient, safety-constrained algorithms, extensive simulation pretraining, and careful reward and environment design before any physical deployment.

Python Code:

```
def safe_action(action, min_bound, max_bound):
    """Clip an RL action to physically safe torque/velocity limits."""
    return max(min_bound, min(action, max_bound))

def domain_randomized_env(base_friction, noise_std=0.05):
    import random
    return base_friction + random.gauss(0, noise_std)

proposed_action = 15.0
safe = safe_action(proposed_action, min_bound=-5.0, max_bound=5.0)
print("Requested action:", proposed_action, "-> Safety-clipped action:", safe)

for episode in range(3):
    friction = domain_randomized_env(base_friction=0.4)
    print(f"Episode {episode}: randomized friction = {friction:.3f}")
```

Output:

```
Requested action: 15.0 -> Safety-clipped action: 5.0
Episode 0: randomized friction = 0.438
Episode 1: randomized friction = 0.408
Episode 2: randomized friction = 0.517
```

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

(5 marks)

Answer:

The discount factor gamma (0<=gamma<=1) controls how strongly future rewards are weighted relative to immediate ones in the return G_t = R_(t+1) + gamma*R_(t+2) + gamma^2*R_(t+3) + ...

A gamma close to 0 makes the agent myopic: it optimises almost entirely for immediate reward and largely ignores long-term consequences, which can lead to short-sighted, greedy behaviour. A gamma close to 1 makes the agent far-sighted, giving nearly equal weight to distant future rewards, encouraging strategic, long-horizon planning - but it also makes the return sum harder to estimate and can slow convergence or increase variance.

In episodic tasks with a well-defined terminal state, gamma can safely be set close to (or exactly) 1 because the sum of rewards is finite by construction. In continuing (non-terminating) tasks, gamma must be strictly less than 1 for the infinite sum of returns to remain finite and mathematically well-defined.

Python Code:

```
def compute_return(rewards, gamma):
    G = 0
    for t, r in enumerate(rewards):
        G += (gamma ** t) * r
    return G

rewards = [1, 1, 1, 1, 10]  # small rewards now, big reward later

for gamma in [0.0, 0.5, 0.9, 0.99]:
    print(f"gamma={gamma}: return = {compute_return(rewards, gamma):.3f}")
```

Output:

```
gamma=0.0: return = 1.000
gamma=0.5: return = 2.500
gamma=0.9: return = 10.000
gamma=0.99: return = 13.546
```

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. (5 marks)

Answer:

Model-free RL (e.g., Q-learning, SARSA, policy gradients) learns a value function or policy directly from experience without ever building an explicit model of the environment's transition and reward dynamics. It is simple to implement and scales well to complex, high-dimensional environments, but it is typically sample-inefficient - it needs a large number of interactions to converge because every update relies purely on observed transitions.

Model-based RL first learns (or is given) a model of the environment's transition and reward functions, and then uses that model for planning (e.g., via simulated rollouts, Monte Carlo Tree Search, or dynamic programming). This is far more sample-efficient because the agent can 'imagine' many outcomes without needing real interactions, and it adapts quickly to changes if the model is accurate.

In dynamic (fast-changing) environments, model-based RL's advantage in sample efficiency and adaptability is offset by the difficulty of learning an accurate model of highly non-stationary dynamics - an imperfect model compounds errors during planning. Model-free RL, though slower to learn, is generally more robust to model-misspecification since it never depends on the correctness of a learned dynamics model. In practice, hybrid approaches (e.g., Dyna-Q) combine both to get the sample efficiency of model-based planning with the robustness of model-free learning.

Python Code:

```
# Model-free style update (Q-learning)
# Learns purely from experience
def q_learning_update(Q, s, a, r, s_next, alpha=0.1, gamma=0.9):
    best_next = max(Q[s_next].values())
    Q[s][a] += alpha * (
        r + gamma * best_next - Q[s][a]
    )
    return Q
# Model-based style planning
# Uses a transition model to simulate future outcomes
def model_based_planning(model, V, states, actions, gamma=0.9, sweeps=20):
    for _ in range(sweeps):
        for s in states:
            V[s] = max(
                sum(
                    p * (
                        model["R"][s][a]
                        + gamma * V[s_next]
                    )
                    for s_next, p in model["P"][s][a].items()
                )
                for a in actions
            )
    return V
# - - - - - - - - - - - - - - - - - -
# Q-Learning Example
# - - - - - - - - - - - - - - - - - -
Q = {
    "S0": {"left": 0.0, "right": 0.0},
    "S1": {"left": 0.0, "right": 0.0},
    "S2": {"left": 0.0, "right": 0.0}
}
Q = q_learning_update(
    Q,
    "S1",
    "right",
    1,
    "S2"
)
```

```
print("Q-Learning Update")
print("----------")
print("State  : S1")
print("Action : right")
print("Reward : 1")
print("Next State : S2")
print("Updated Q-value :", round(Q["S1"]["right"], 3))
# ----------
# Model-Based Planning Example
# ----------
model = {
    "P": {
        "S0": {
            "left": {"S0": 1.0},
            "right": {"S1": 1.0}
        },
        "S1": {
            "left": {"S0": 0.8, "S2": 0.2},
            "right": {"S2": 1.0}
        },
        "S2": {
            "left": {"S1": 1.0},
            "right": {"S2": 1.0}
        }
    },

    "R": {
        "S0": {"left": 0, "right": 0},
        "S1": {"left": 0, "right": 1},
        "S2": {"left": 0, "right": 5}
    }
}

states = ["S0", "S1", "S2"]
actions = ["left", "right"]
V = {
    "S0": 0.0,
    "S1": 0.0,
    "S2": 0.0
}
V = model_based_planning(
    model,
    V,
    states,
    actions
)
print("\nModel-Based Planning")
print("----------")
```

```
for state in states:
    print(state, "Value :", round(V[state], 3))
```

Output:

```
Q-Learning Update
-----------------
State  : S1
Action : right
Reward : 1
Next State : S2
Updated Q-value : 0.1

Model-Based Planning
--------------------
S0 Value : 35.321
S1 Value : 39.921
S2 Value : 43.921
```

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. (5 marks)

Answer:

Q-learning is an off-policy, model-free algorithm that learns the optimal action-value function Q*(s,a) directly, regardless of the policy being followed during exploration. After each transition (s, a, r, s'), it updates Q(s,a) using the Bellman optimality update:

Q(s,a) <- Q(s,a) + alpha * [r + gamma * max_a' Q(s',a') - Q(s,a)]

Here alpha is the learning rate and the term in brackets is the temporal-difference (TD) error - the difference between the current estimate and a better bootstrapped estimate using the observed reward and the best value of the next state. Over many episodes, as every state-action pair is visited sufficiently often and alpha is annealed appropriately, Q converges to Q* with probability 1 (guaranteed under standard stochastic-approximation conditions), after which the greedy policy with respect to Q becomes optimal.

In practice, convergence shows up as the TD error shrinking toward zero and the estimated Q-values stabilising across episodes, as illustrated in the example below on a tiny 1-D 'chain' gridworld.

Python Code:

```
import random
```

```
n_states = 6          # states 0..5, 5 is the goal
n_actions = 2         # 0 = left, 1 = right
Q = [[0.0, 0.0] for _ in range(n_states)]
alpha, gamma, epsilon = 0.1, 0.95, 0.2

def step(state, action):
    next_state = state - 1 if action == 0 else state + 1
    next_state = max(0, min(n_states - 1, next_state))
    reward = 1.0 if next_state == n_states - 1 else -0.01
    done = next_state == n_states - 1
    return next_state, reward, done

for episode in range(500):
    state = 0
    done = False
    while not done:
        if random.random() < epsilon:
            action = random.randrange(n_actions)
        else:
            action = Q[state].index(max(Q[state]))
        next_state, reward, done = step(state, action)
        td_target = reward + gamma * max(Q[next_state])
        Q[state][action] += alpha * (td_target - Q[state][action])
        state = next_state

print("Learned Q-table:")
for s, vals in enumerate(Q):
    print(f"state {s}: left={vals[0]:.3f}, right={vals[1]:.3f}")
```

Output:

```
Learned Q-table:
state 0: left=0.727, right=0.777
state 1: left=0.728, right=0.829
state 2: left=0.777, right=0.883
state 3: left=0.822, right=0.940
state 4: left=0.877, right=1.000
state 5: left=0.000, right=0.000
```

10 Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. (5 marks)

Answer:

On-policy methods (e.g., SARSA) learn the value of the policy that the agent is actually using to select actions, including its exploratory behaviour. Off-policy methods (e.g., Q-learning) learn the value of a different target policy (usually the greedy/optimal one) than the behaviour policy used to generate the data.

SARSA's update uses the action actually taken next, A_(t+1): Q(s,a) <- Q(s,a) + alpha*[r + gamma*Q(s',a') - Q(s,a)]. Q-learning's update instead uses the maximum over all next actions: Q(s,a) <- Q(s,a) + alpha*[r + gamma*max_a' Q(s',a') - Q(s,a)], regardless of which action is actually taken next.

On-policy methods tend to be more conservative and safer during learning because they account for the exploratory (e.g., epsilon-greedy) actions the agent might still take, making them well suited to environments where mistakes during exploration are costly (e.g., near cliffs/hazards). Off-policy methods are typically more sample-efficient because they can reuse old data / experience replay and learn about the optimal policy even while exploring randomly or using a completely different behaviour policy, but they can be less stable and are more prone to overestimation bias without techniques such as Double Q-learning.

Python Code:

```
import random

def sarsa_update(Q, s, a, r, s_next, a_next, alpha=0.1, gamma=0.9):
    # ON-POLICY: uses the action actually taken next (a_next)
    Q[s][a] += alpha * (r + gamma * Q[s_next][a_next] - Q[s][a])
    return Q

def q_learning_update(Q, s, a, r, s_next, alpha=0.1, gamma=0.9):
    # OFF-POLICY: uses the greedy/best action regardless of what is taken next
    Q[s][a] += alpha * (r + gamma * max(Q[s_next]) - Q[s][a])
    return Q

# Both algorithms can be run on the same transition to compare their targets
Q_sarsa = [[0.0, 0.0] for _ in range(5)]
Q_qlearn = [[0.0, 0.0] for _ in range(5)]

s, a, r, s_next, a_next = 1, 1, -0.01, 2, 0
Q_sarsa = sarsa_update(Q_sarsa, s, a, r, s_next, a_next)
Q_qlearn = q_learning_update(Q_qlearn, s, a, r, s_next)

print("SARSA (on-policy) Q(1,1):", Q_sarsa[1][1])
print("Q-learning (off-policy) Q(1,1):", Q_qlearn[1][1])
```

Output:

```
SARSA (on-policy) Q(1,1): -0.001
Q-learning (off-policy) Q(1,1): -0.001
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately